2.4.2 Resolution Algorithm for Qualified Constants

Qualified constants look like this:

```
Billing::Invoice
```

`Billing::Invoice` is composed of two constants: `Billing` is relative and is resolved using the algorithm of the previous section.
Leading colons would make the first segment absolute rather than relative: `::Billing::Invoice`. That would force `Billing` to be looked up only as a top-level constant.

`Invoice` on the other hand is qualified by `Billing` and we are going to see its resolution next. Let's define *parent* to be that qualifying class or module object, that is, `Billing` in the example above. The algorithm for qualified constants goes like this:

1. The constant is looked up in the parent and its ancestors.

2. If the lookup fails, `const_missing` is invoked in the parent. The default implementation of `const_missing` raises `NameError`, but it can be overridden.

As you see, this algorithm is simpler than the one for relative constants. In particular, the nesting plays no role here, and modules are not special-cased, if neither they nor their ancestors have the constants, `Object` is **not** checked.
Rails autoloading **does not emulate this algorithm**, but its starting point is the name of the constant to be autoloaded, and the parent. See more in Qualified References.

3 Vocabulary

3.1 Parent Namespaces

Given a string with a constant path we define its *parent namespace* to be the string that results from removing its rightmost segment.
For example, the parent namespace of the string "A::B::C" is the string "A::B", the parent namespace of "A::B" is "A", and the parent namespace of "A" is "".

The interpretation of a parent namespace when thinking about classes and modules is tricky though. Let's consider a module M named "A::B":

- The parent namespace, "A", may not reflect nesting at a given spot.

- The constant `A` may no longer exist, some code could have removed it from `Object`.
- If `A` exists, the class or module that was originally in `A` may not be there anymore. For example, if after a constant removal there was another constant assignment there would generally be a different object in there.
- In such case, it could even happen that the reassigned `A` held a new class or module called also "A"!
- In the previous scenarios M would no longer be reachable through `A::B` but the module object itself could still be alive somewhere and its name would still be "A::B".

The idea of a parent namespace is at the core of the autoloading algorithms and helps explain and understand their motivation intuitively, but as you see that metaphor leaks easily. Given an edge case to reason about, take always into account that by "parent namespace" the guide means exactly that specific string derivation.